Source code:

```
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, classification_report

# 1: Data generation and preparation
# Loads the Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Describes the dataset's characteristics: 150 samples, 4 features, 3
classes
print(f"Features: {iris.feature_names}")
print(f"Species: {iris.target_names}")

# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
random_state = 42)

# Rule-Based classifier function
def rule_based_predict(features):
    predictions = []
    for f in features:
        # petal length is index 2
        if f[57] < 2.45:
            predictions.append(0) # Setosa
        elif f[58] > 1.75:
            predictions.append(2) # Virginica
        else:
            predictions.append(1) # Versicolor
    return predictions
```

```
# 2 & 3: Model Development and Evaluation
models = {
    "Decision Tree": DecisionTreeClassifier(),
    "Naïve Bayes": GaussianNB(),
    "Logistic Regression": LogisticRegression(max_iter = 200),
    "KNN": KNeighborsClassifier(n_neighbors = 3),
    "SVM": SVC(kernel = 'linear')
}

print("\nModel Evaluation Results")

# Evaluates ML models
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"\nModel: {name}")
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print(classification_report(y_test, y_pred,
target_names=iris.target_names))

# Evaluates Rule-Based Classifier separately
y_rule_pred = rule_based_predict(X_test)
print("\nModel: Rule-Based Classifier")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_rule_pred))
print(classification_report(y_test, y_rule_pred,
target_names=iris.target_names))
```

Output:

```
Model Evaluation Results

Model: Decision Tree
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


Model: Naïve Bayes
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


Model: Logistic Regression
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

```
Model: KNN
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


Model: SVM
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

1. Data Generation and Preparation

The Iris dataset is a key pillar of machine learning classification. It represents three species of flowers: Iris Setosa, Iris Versicolor, and Iris Virginica (Data Professor, 2020). The dataset contains 150 samples, with 50 samples dedicated to each class to balance the class distribution. Each sample includes the features of sepal length, sepal width, petal length, and petal width.

To prepare the data, it is assigned to the input variable (X) that contains the four aforementioned characteristics, and a target variable (Y) that contains the species labels. The train-test split is a crucial step in the workflow responsible for dividing the data into a training set to build the model and a testing set for evaluation. A common ratio is an 80/20 split, where 80% of the data is used for learning, and the remaining 20% is reserved for assessing generalization to unseen data (Data Professor, 2020).

2. Model Development

2A.

Decision trees are supervised algorithms that use a hierarchical structure to partition data into subsets (Sharma, 2026). They function as a series of if-then statements to create leaf nodes where the most data points belong to the same class. For example, “Is petal length > 2.45?” They offer high interpretability but are susceptible to overfitting if the tree becomes too complex (Sharma, 2026).

A rule-based classifier represents knowledge through explicit logical rules (Bansal, 2024). In the given scenario, a manual rule-set can be derived. For example, if petal length is less than 2cm, the species is Setosa. While these models offer fast execution and are simple to interpret, they may not be exhaustive, which could leave some samples unclassified unless a default class is assigned (Bansal, 2024).

Naive Bayes is a probabilistic classifier based on Bayes’ Theorem that estimates the posterior probability of a class given various predictors (Ray, 2025). It is called “naive” because it makes the simplistic assumption that all features are independent, even when they are actually related in real-world scenarios. Despite the unrealistic assumptions, it is very fast and performs

well on many real-world datasets, particularly where categorical variables are concerned (Ray, 2025).

2B.

Logistic regression predicts the probability of an input belonging to a specific class. It utilizes a sigmoid function to map continuous values to a range between 0 and 1 (kartik, 2026). For multiclass problems, such as the Iris dataset, the multinomial logistic regression extension can be applied to predict one of the three species.

K-Nearest Neighbors (KNN) is an algorithm that stores training data and classifies new points based on their proximity to existing ones (Kavlakoglu, n.d.). It identifies the nearest neighbors “k” using Euclidean distance and assigns a class label based on a majority vote (Infinite Codes, 2024). Its ease of implementation comes at the cost of becoming increasingly computationally expensive as the dataset grows.

Support Vector Machines (SVMs) work by finding the optimal hyperplane that maximizes the margin between classes (Infinite Codes, 2024). For non-linearly separable data, the SVM uses the kernel trick to map the original features into a higher-dimensional space where a linear boundary can be established. The support vectors are data points closest to the boundary (Sasidharan, 2026).

3. Evaluation Metrics

To assess model performance, three primary metrics are used. They are a confusion matrix, precision/recall, and F1 score. A confusion matrix shows the model’s performance by breaking down true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) into a table (Mahajan, 2026). Precision measures the accuracy of positive predictions,

which is the ratio of TP to all positive predictions. Recall measures the ability of the model to identify all actual positive cases. F1 score is the harmonic mean of precision and recall (Mahajan, 2026). It provides a single score that balances the trade-off between the two, which is useful for judging the model's overall robustness.

4. Analysis and Comparison

Regarding the Iris dataset, distance-based and optimization-based models, such as SVM and logistic regression, normally achieve almost perfect accuracy. This is due to the dataset's high class separability, notably in the Setosa species, which is linearly separable from the other two (Data Professor, 2020).

The best-performing model is either SVM or KNN. SVM excels at creating clear decision boundaries in the Iris dataset's relatively low-dimensional space (Sasidharan, 2026). KNN also performs well in this scenario because the features are continuous and the clusters for each species are physically close together in the feature space.

The characteristics of the dataset have a substantial influence on the model's performance. This dataset avoids the curse of dimensionality due to only having four features. This allows KNN to remain accurate and efficient (Kavlakoglu, n.d.). Additionally, the linear relationship between petal length/width and the species type makes logistic regression a stable and effective option. On the other hand, the Naive Bayes model might struggle a little more than the others because petal length and width are often strongly correlated, which throws a wrench into the assumption of feature independence.

Word count: 815

References

Bansal, N. (2024, November 18). *What is rule-based data mining classifier? Explained with examples*. Hevo Data. https://hevodata.com/learn/rule-based-data-mining/

Data Professor. (2020, March 11). *Machine learning in Python: Building a classification model* [Video]. YouTube. https://www.youtube.com/watch?v=XmSlFPDjKdc

Infinite Codes. (2024, September 17). *All machine learning algorithms explained in 17 min* [Video]. YouTube. https://www.youtube.com/watch?v=E0Hmnixke2g

kartik. (2026, June 1). *Logistic Regression in machine learning*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/

Kavlakoglu, E. (n.d.). *What is the k-nearest neighbors algorithm?* IBM. https://www.ibm.com/think/topics/knn

Mahajan, R. (2026, March 30). *F1 score in machine learning*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/f1-score-in-machine-learning/

Ray, S. (2025, May 1). *Naive Bayes classifier explained with practical problems*. Analytics Vidhya. https://www.analyticsvidhya.com/blog/2017/09/naive-bayes-explained/

Sasidharan, A. (2026, May 2). *Support Vector Machine (SVM) algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/

Sharma, A. (2026, May 2). *Decision Tree in machine learning*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/decision-tree-introduction-example/